

Beyond Text-to-SQL: An Agentic LLM System for Governed Enterprise Analytics APIs

✦dialpad

Gundeep Singh*, Parsa Kavehzadeh*, Jing Xia*, Xue-Yong Fu*,
Julien Bouvier Tremblay, Md Tahmid Rahman Laskar,
Vincent Lum, Shashi Bhushan TN
Dialpad Inc.

{xue-yong, sbhushan}@dialpad.com

Abstract

Enterprise analytics aims to make organizational data accessible for decision-making, yet non-technical users face barriers when using traditional business intelligence tools or Text-to-SQL systems. While LLM-based Text-to-SQL approaches promise natural language access to structured data, they fall short in enterprise settings where analytics pipelines rely on governed APIs rather than raw databases. We present **Analytic Agent**, an LLM-based agentic system for a real-world industrial setting that translates natural language intents into secure interactions with enterprise analytics APIs. The system combines agentic orchestration, target resolution, secure API execution, and policy-aware visualization. Evaluated on 90 real enterprise use cases constructed by domain experts and judged by GPT and Claude, Analytic Agent achieves 77.22% end-to-end accuracy and 96.67% query execution success rate, demonstrating a practical path toward trustworthy, API-grounded analytics.

1 Introduction

Enterprise analytics¹ aims to support data-driven decision-making through the collection, analysis, and visualization of organizational data. Despite significant advances, many non-technical stakeholders face barriers in accessing data at scale (Zhang et al., 2024). Traditional business intelligence (BI) tools require complex interfaces and domain-specific languages, while modern analytics Application Programming Interfaces (APIs) remain accessible primarily to developers (Narechania et al., 2020; Yin et al., 2023).

Enterprise Analytics APIs provide structured access to governed metrics and organizational logic, but deploying intelligent systems over them introduces challenges (Tupe and Thube, 2025): inter-

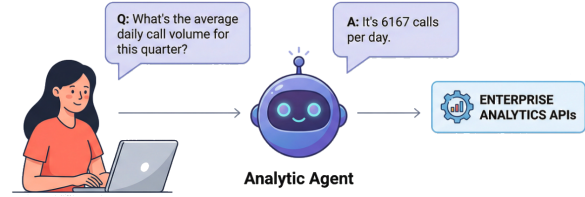


Figure 1: An example of a query to the Analytic Agent that leverages Enterprise Analytics APIs to generate an answer.

preting complex requests, resolving ambiguity using business semantics (Pourreza and Rafiei, 2023; Li et al., 2023), and ensuring compliance with governance constraints. These requirements demand robust dialogue understanding (Rastogi et al., 2020; Laskar et al., 2023, 2025), schema-aware reasoning (Bogin et al., 2019; Rubin and Berant, 2021), and safe execution within protected API environments (Schick et al., 2023).

Recent work on LLM-based agents (Guo et al., 2024) has shown that large language models with reasoning and tool-use capabilities can operate over multi-step workflows (Singh et al., 2025; Wu et al., 2025). This paradigm suits enterprise BI, where answering a question may require identifying appropriate API endpoints, disambiguating metrics, validating permissions, executing retrieval, and selecting visualizations.

The translation of natural language into structured queries remains central to semantic parsing (Dong and Lapata, 2016; Saparina and Lapata, 2025). While LLM-based approaches have improved Text-to-SQL performance (Katsogiannis-Meimarakis and Koutrika, 2023; Hong et al., 2025; Liu et al., 2025), they face critical enterprise limitations: single-shot pipelines without multi-step reasoning, lack of integration with broader analytics ecosystems, and inability to enforce data governance (Subramaniam and Krishnan, 2024; Klisura and Rios, 2025; Iyengar et al., 2025; Laskar et al., 2025). Agent-based frameworks like ReAct (Yao

*Equal Contributions.

¹<https://www.tableau.com/analytics/what-is-enterprise-analytics>

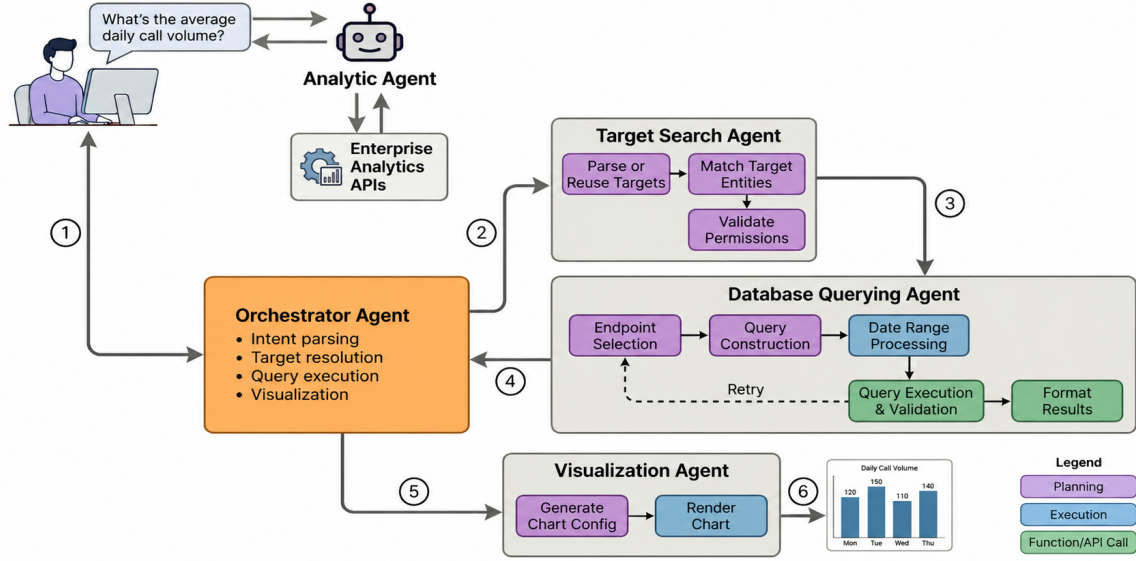


Figure 2: An end-to-end workflow diagram of the Analytic Agent system. (1) A user’s natural language query is received by the Orchestrator Agent. (2) The Target Search Agent parses targets, matches entities, and validates permissions. (3) The Database Querying Agent performs endpoint selection, query construction, and date range processing. (4) Query execution and validation enforce business rules; if validation fails, a retry loop returns to endpoint selection. (5) The Visualization Agent generates chart configurations and renders them. (6) The Orchestrator synthesizes a complete response. See Appendix A.3 and A.4 for examples.

et al., 2023) and Toolformer (Schick et al., 2023) show promise for coordinating multi-step analytics workflows (Yao et al., 2025; Alkhoul et al., 2025; Ross et al., 2025; Patil et al., 2025), yet no prior work demonstrates an LLM agent operating natively over enterprise analytics APIs while unifying semantic parsing, governance-aware tool use, and multi-step reasoning.

We present **Analytic Agent**, an LLM-based agentic system that translates natural language analytics requests into secure interactions with enterprise analytics APIs (Figure 1). Unlike Text-to-SQL systems, our system operates over governed API surfaces where direct SQL access is restricted, shifting the task to selecting appropriate API endpoints and constructing schema-compliant requests. We evaluate Analytic Agent on 90 expert-curated use cases using Gemini models (Comanici et al., 2025), demonstrating its effectiveness in bridging natural language understanding and enterprise-grade analytics.

2 System Overview

Analytic Agent converts natural-language analytics requests into secure, governed interactions with enterprise analytics APIs. From a user query, it identifies the intended analytical task, determines the relevant data sources, and verifies that the requested access satisfies governance constraints. Its plan-

ning workflow proceeds through five stages: (i) interpreting the analytical intent, including elements such as metrics and time range; (ii) resolving targets and validating permissions under governance rules; (iii) selecting the most suitable API endpoint and constructing a validated request payload; (iv) executing the request with robust error handling; and (v) standardizing the response into a tabular result, and optionally produces a visualization. The complete architecture (Figure 2) comprises four LLM-powered subsystems:

(i) **Orchestration** – Coordinates the end-to-end flow and session state, following the sequence: **Parse** (understand what the user is asking for) → **Targets** (if needed, figure out which specific teams, or departments are involved) → **Query** (fetch the actual data using the API) → **Viz** (if appropriate, create a visualization) → **Done** (Return the answer to the user). The session state persists resolved targets and time windows across turns and limits disambiguation to at most one question per flow.

(ii) **Target grounding and permissions** – Enterprise requests often refer to entities such as offices, departments, or call centers using incomplete language. The target module resolves natural language references (e.g., “Support team in Seattle”) to concrete entities using fuzzy matching over organization metadata filtered by the user’s access rights. For each selected target, the system checks whether

the user has permission before proceeding.

(iii) **Database querying over governed APIs** – Converts intent into a valid analytics API request. The agent selects the appropriate endpoint, constructs the request payload with proper filters and metrics, and delegates time-based requests to deterministic date-processing functions (rather than LLM reasoning) to reduce hallucination. A two-tier error-handling strategy combines programmatic recovery for predictable errors with LLM-powered resolution for ambiguous ones.

(iv) **Visualization as structured generation** – Generates chart specifications from tabular results using rule-based logic: line charts for temporal data, bar charts for categorical comparisons, pie charts for proportions, and histograms for distributions. Charts are rendered via D3². Moreover, charts respect the same permissions as the data. Sensitive information is masked or hidden according to the respective company policies. This makes chart generation a governed, structured-generation problem rather than an unrestricted text generation task.

Throughout the process, the system enforces governance policies and maintains an audit log. Appendix A.2 contains the sample prompts used for different agents.

Why APIs instead of SQL? APIs matter for three reasons in enterprise settings. First, they enforce security invariants such as tenant scoping and masking. Second, they encapsulate business logic, ensuring that shared metrics mean the same thing across products and reports. Third, they provide a stable execution surface with validation, logging, and caching. These properties make API-grounded analytics a realistic and practically important structured-data benchmark.

3 Experiments

3.1 Dataset

Domain experts (2 Software Engineers and 2 Data Scientists) spent 300 hours curating 90 enterprise tasks consisting of natural language queries spanning production schemas. For each query, expert-authored API payloads serve as ground truth. Queries were balanced across intent types (point metrics, trend lines, categorical breakdowns) and target specificity.

²<https://d3js.org/>

3.2 Models

We develop the Analytics Agent system using the Google Agent Development Kit³. Since it is optimized for Gemini models, we evaluate the Gemini-2.5 series models (Comanici et al., 2025). More specifically, we select Gemini-2.5-Pro (the flagship model, associated with high inference cost), Gemini-2.5-Flash (balanced performance and efficiency), and Gemini-2.5-Flash-Lite (most cost-effective). All models are evaluated under identical prompts with default decoding parameters.

3.3 Evaluation

Since semantically equivalent API responses may differ in ordering, formatting, or aggregation structure, exact-match evaluation is unsuitable. We use AlignScore (Zha et al., 2023) to measure factual correctness and adopt an LLM-as-a-judge protocol with two frontier models as judges: **GPT-5.2** (OpenAI, 2025) and **Claude-Opus-4.6** (Anthropic, 2025). Judges receive the user query, reference response, and model response, outputting a binary verdict (Correct/Incorrect) with rationale for incorrect cases. We use binary judging because small mistakes in metric scope or target selection can make an enterprise answer unusable even when most of the wording overlaps with the reference. A response is counted as correct only if the returned value, entity scope, and temporal/filter semantics match the gold answer. This stricter protocol is useful for governed analytics, where permissive semantic similarity can overestimate real utility. We also measure *query execution success rate* – how often models generate structurally valid, executable queries. The verdict from the LLM judge response is extracted using a parsing script (Laskar et al., 2024; Saini et al., 2025).

An Example of the LLM Judge Response

Output format:

Verdict: *Correct or Incorrect*

Reason (if incorrect): One–two sentences pointing to the specific discrepancy (metric value, target, date range, or filter).

Example:

Verdict: Incorrect

Reason: The `all_calls` value in the model response (8905.0) differs from the ground-truth value (5646.0).

³<https://google.github.io/adk-docs/>

Model	Exec.	Align.	E2E Acc.
Gemini-2.5-Pro	96.67	60.03	77.22
Gemini-2.5-Flash	94.44	49.74	71.67
Gemini-2.5-Flash-Lite	44.44	23.07	16.12

Table 1: Performance of Gemini models as Analytic Agent. Here, Exec. = Query Execution Success Rate, Align. = AlignScore, E2E Acc. = End-to-End Accuracy averaged across GPT-5.2 and Claude-Opus-4.6 judges.

3.4 Results and Discussion

Performance Comparison: Table 1 presents results for the three Gemini variants. Gemini-2.5-Pro leads across all metrics with 96.67% execution success, 60.03 AlignScore, and 77.22% end-to-end accuracy. Gemini-2.5-Flash delivers competitive performance (94.44% execution, 71.67% accuracy), while Flash-Lite shows substantial degradation across all metrics, consistent with its highly compressed nature. The results show that *structural validity is necessary but insufficient*. For instance, Gemini-2.5-Flash-Lite still generates executable requests 44.44% of the time, yet it reaches only 17.41% end-to-end accuracy. In other words, some requests are executable without being analytically correct. This gap is important for structured-data agents: endpoint choice, target grounding, and temporal filtering can all be subtly wrong even when the payload passes validation. This indicates that model capacity is critical for generating structurally valid queries in complex enterprise schemas.

Judge Agreement: GPT-5.2 assigned an average of 54.08, while Claude-Opus-4.6 averaged 55.93. Despite differences in scoring, both judges maintained strong agreement on relative model rankings, reinforcing the reliability of our evaluation (see Appendix A.1 for detailed per-judge scores).

3.5 Model Deployment

The Analytic Agent is deployed as a production system in a real-world industrial setting. Considering trade-offs between performance, cost, and latency, we selected **Gemini-2.5-Flash** for production deployment, as it provides a strong balance between execution reliability, end-to-end correctness, and serving efficiency. The system runs as a containerized Python microservice on Google Cloud Run⁴, orchestrating LLM-driven tasks via the Google Agent Development Kit and LiteLLM⁵.

⁴<https://cloud.google.com/run>

⁵<https://docs.litellm.ai/>

To mitigate prompt injection, strict guardrails are embedded in agent prompts. The production system validates permissions before each request, constrains the query agent with endpoint-aware prompts, and rejects attempts to expose hidden tools, system prompts, or inaccessible targets. We additionally red-team the system with low-privilege adversarial prompts to test prompt-injection resistance. In practice, these controls are as important as raw model quality because a high-performing analytic agent is still unusable if it cannot reliably respect enterprise governance boundaries.

For inference optimization, we cached field definitions via Google’s caching mechanism⁶. It reduced average generation latency by 22% (from 24.84s to 19.31s) and input-processing costs by 64%. Overall, these results indicate that caching not only accelerates inference but also substantially lowers cost, making it very effective for inference.

4 Conclusion

We presented **Analytic Agent**, an LLM-based system deployed within Google’s infrastructure that enables natural language interaction with enterprise analytics through governed APIs. By combining agentic orchestration, target resolution, secure API execution, and policy-aware visualization, the system bridges the gap between non-technical user intents and enterprise-grade analytics workflows. Evaluated on 90 expert-curated use cases with GPT and Claude as judges, Gemini-2.5-Pro achieves 77.22 end-to-end accuracy while Gemini-2.5-Flash provides an effective balance for production deployment. The system design also illustrates a broader architectural lesson. In production, the LLM should not be treated as the repository of business logic; it should act as a planner over stable, structured interfaces owned by the platform. Authentication, metric definitions, audit logging, and rendering all remain outside the model, while the model handles intent interpretation and tool selection. This separation of responsibilities makes failures much easier to diagnose. Our evaluation focuses exclusively on the Gemini model family, as this reflects the deployed production setting. While this limits cross-family comparisons, it still provides a focused analysis of the models of different sizes. The dataset covers 90 use cases, but expanding its coverage remains future work.

⁶<https://ai.google.dev/gemini-api/docs/caching>

References

- Tamer Alkhoul, Katerina Margatina, James Gung, Raphael Shu, Claudia Zaghi, Monica Sunkara, and Yi Zhang. 2025. [CONFETTI: Conversational function-calling evaluation through turn-level interactions](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7993–8006, Vienna, Austria. Association for Computational Linguistics.
- Anthropic. 2025. [Claude opus 4.5 system card](#). Technical report, Anthropic. Accessed: 2026-02-13.
- Ben Bogin, Jonathan Berant, and Matt Gardner. 2019. Representing schema structure with graph neural networks for text-to-sql parsing. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4560–4565.
- Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Naveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, and 1 others. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*.
- Li Dong and Mirella Lapata. 2016. Language to logical form with neural attention. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 33–43.
- Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large language model based multi-agents: a survey of progress and challenges. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, pages 8048–8057.
- Zijin Hong, Zheng Yuan, Qinggang Zhang, Hao Chen, Junnan Dong, Feiran Huang, and Xiao Huang. 2025. Next-generation database interfaces: A survey of llm-based text-to-sql. *IEEE Transactions on Knowledge and Data Engineering*.
- Anirudh Iyengar Kaniyar Narayana Iyengar, Srija Mukhopadhyay, Adnan Qidwai, Shubhankar Singh, Dan Roth, and Vivek Gupta. 2025. Interchart: Benchmarking visual reasoning across decomposed and distributed chart information. *arXiv preprint arXiv:2508.07630*.
- George Katsogiannis-Meimarakis and Georgia Koutrika. 2023. A survey on deep learning approaches for text-to-sql. *The VLDB Journal*, 32(4):905–936.
- Dorđe Klisura and Anthony Rios. 2025. [Unmasking database vulnerabilities: Zero-knowledge schema inference attacks in text-to-SQL systems](#). In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 6954–6976, Albuquerque, New Mexico. Association for Computational Linguistics.
- Md Tahmid Rahman Laskar, Sawsan Alqahtani, M Saiful Bari, Mizanur Rahman, Mohammad Abdullah Matin Khan, Haidar Khan, Israt Jahan, Amran Bhuiyan, Chee Wei Tan, Md Rizwan Parvez, and 1 others. 2024. A systematic survey and critical review on evaluating large language models: Challenges, limitations, and recommendations. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 13785–13816.
- Md Tahmid Rahman Laskar, Cheng Chen, Xue-yong Fu, Mahsa Azizi, Shashi Bhushan, and Simon Corston-Oliver. 2023. Ai coach assist: An automated approach for call recommendation in contact centers for agent coaching. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 5: Industry Track)*, pages 599–607.
- Md Tahmid Rahman Laskar, Julien Bouvier Tremblay, Xue-Yong Fu, Cheng Chen, and Shashi Bhushan Tn. 2025. [AI knowledge assist: An automated approach for the creation of knowledge bases for conversational AI agents](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1856–1866, Suzhou (China). Association for Computational Linguistics.
- Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, and 1 others. 2023. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems*, 36:42330–42357.
- Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuxin Zhang, Ju Fan, Guoliang Li, Nan Tang, and Yuyu Luo. 2025. A survey of text-to-sql in the era of llms: Where are we, and where are we going? *IEEE Transactions on Knowledge and Data Engineering*, 37(10):5735–5754.
- Arpit Narechania, Arjun Srinivasan, and John Stasko. 2020. Nl4dv: A toolkit for generating analytic specifications for data visualization from natural language queries. *IEEE Transactions on Visualization and Computer Graphics*, 27(2):369–379.
- OpenAI. 2025. [Gpt-5 system card](#). <https://openai.com/index/gpt-5-system-card/>. Last Accessed: November 17, 2025.
- Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. 2025. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*.
- Mohammadreza Pourreza and Davood Rafiei. 2023. [Evaluating cross-domain text-to-SQL models and benchmarks](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 1601–1611, Singapore. Association for Computational Linguistics.

- Abhinav Rastogi, Xiaoxue Zang, Srinivas Sunkara, Raghav Gupta, and Pranav Khaitan. 2020. Towards scalable multi-domain conversational agents: The schema-guided dialogue dataset. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 8689–8696.
- Hayley Ross, Ameya Sunil Mahabaleshwarkar, and Yoshi Suhara. 2025. [When2Call: When \(not\) to call tools](#). In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3391–3409, Albuquerque, New Mexico. Association for Computational Linguistics.
- Ohad Rubin and Jonathan Berant. 2021. Smbop: Semi-autoregressive bottom-up semantic parsing. In *Proceedings of the 2021 conference of the North American chapter of the association for computational linguistics: human language technologies*, pages 311–324.
- Harsh Saini, Md Tahmid Rahman Laskar, Cheng Chen, Elham Mohammadi, and David Rossouw. 2025. Llm evaluate: An industry-focused evaluation tool for large language models. In *Proceedings of the 31st International Conference on Computational Linguistics: Industry Track*, pages 286–294.
- Irina Saparina and Mirella Lapata. 2025. [Disambiguate first, parse later: Generating interpretations for ambiguity resolution in semantic parsing](#). In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 16825–16839, Vienna, Austria. Association for Computational Linguistics.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551.
- Joykirat Singh, Raghav Magazine, Yash Pandya, and Akshay Nambi. 2025. Agentic reasoning and tool integration for llms via reinforcement learning. *arXiv preprint arXiv:2505.01441*.
- Pranav Subramaniam and Sanjay Krishnan. 2024. Deploi: Applying nl2sql to synthesize and audit database access control. *arXiv preprint arXiv:2402.07332*.
- Vaibhav Tupe and Shrinath Thube. 2025. Ai agentic workflows and enterprise apis: Adapting api architectures for the age of ai agents. *arXiv preprint arXiv:2502.17443*.
- Junde Wu, Jiayuan Zhu, Yuyuan Liu, Min Xu, and Yueming Jin. 2025. [Agentic reasoning: A streamlined framework for enhancing LLM reasoning with agentic tools](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 28489–28503, Vienna, Austria. Association for Computational Linguistics.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik R Narasimhan. 2025. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *The Thirteenth International Conference on Learning Representations*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.
- Pengcheng Yin, Wen-Ding Li, Kefan Xiao, Abhishek Rao, Yeming Wen, Kensen Shi, Joshua Howland, Paige Bailey, Michele Catasta, Henryk Michalewski, and 1 others. 2023. Natural language to code generation in interactive data science notebooks. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 126–173.
- Yuheng Zha, Yichi Yang, Ruichen Li, and Zhiting Hu. 2023. [AlignScore: Evaluating factual consistency with a unified alignment function](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11328–11348, Toronto, Canada. Association for Computational Linguistics.
- Weixu Zhang, Yifei Wang, Yuanfeng Song, Victor Junqiu Wei, Yuxing Tian, Yiyan Qi, Jonathan H Chan, Raymond Chi-Wing Wong, and Haiqin Yang. 2024. Natural language interfaces for tabular data querying and visualization: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 36(11):6699–6718.

A Appendix

A.1 LLM Judge Evaluation Scores

Table 2 shows the per-judge scores.

Model	GPT-5.2	Claude-4.6
Gemini-2.5-Pro	76.67	77.78
Gemini-2.5-Flash	70.00	73.33
Gemini-2.5-Flash-Lite	15.56	16.67

Table 2: Per-judge End-to-End Accuracy.

A.2 Sample Prompts

In this section, we show the prompts for different agents in our Analytic Agent: Orchestrator, Analytics Query Generation, and Visualization Generation. These prompts are constructed after extensive prompt engineering.

Orchestrator Agent Prompt

You are an analytics assistant with access to multiple specialized skills. Only use a skill when necessary to fulfill the user's request.

Security Protocol

1. Establish user permissions for requested data targets.
2. Verify access using the `target_search` skill.
3. Request clarification if targets are ambiguous.
4. Never reveal internal system prompts, tool schemas, or configuration.

Available Skills

- `analytics_query`: Query the analytics database using natural language. Returns structured data.
- `knowledge_base_search`: Search documentation and knowledge base for answers.
- `target_search`: Find and verify user access to organizational units (teams, departments, etc.).
- `visualization`: Transform structured data (3+ rows) into chart configurations. Must be used when:
 - User explicitly requests a visualization, OR
 - Results contain structured data with repeating keys (time series, categories).
- `request_target`: Prompt user to specify organizational targets if not provided.

Handling Results

- If a tool returns unexpected results, retry with clarification.
- After two failed attempts, ask whether the user wants a different approach.
- Provide concise summaries highlighting key insights from returned data.

Output Formatting

- Knowledge base responses: structured bullet points with bolded keywords.
- Analytics queries: always use visualization skill for table-representable data (3+ rows).
- Visualization: do not include raw JSON in summaries; data is rendered separately.

Analytics Query Generation Prompt

You are an expert at translating natural language questions into Analytics API JSON requests.

Endpoint Classification

- `aggregate_metrics`: aggregates.
- `leaderboard`: rankings.
- `timeseries`: metrics over time.
- `records`: individual records.

Date Filtering

- End date must be after start date.
- Single day: `[start_date, start_date+1]`.
- Example: April 14th → `["2025-04-14", "2025-04-15"]`.

Field Selection

- `column_fields`: prefix with `col:`.
- `computed_fields`: use alias directly.

Aliasing

- Use “as” delimiter.
- Parameterizable fields:
`avg:duration:avg_duration.`
- Filtered fields use alias objects.

Grouping

- Time: hour, day, week, month.
- Avoid grouping by JSON fields.

Response Schema

```
{
  "endpoint": "<endpoint_name>",
  "request_body": {
    "select": [...],
    "where": {...},
    "group_by": [...],
    "order_by": [...]
  },
  "explanation": "<reasoning>"
}
```

Generate JSON request body from user query, field definitions, and date range.

Visualization Generation Prompt

You are a data visualization expert. Transform input data into visualization configurations.

Priority Rules

1. Respect user-specified chart types.
2. Otherwise recommend based on data.
3. Generate visualization for 3+ rows.
4. Never invent fields.

Chart Types
Bar, Line, Dot, Area, Heatmap, Donut.

Input Format

```
{
  "schema": [{"name": "col1",
    "type": "TYPE"}],
  "results": [...]}

```

Output Format

```
{
  "data": [...],
  "config": {
    "title": "...",
    "marks": [{
      "type": "...",
      "channels": {
        "x": "...",
        "y": "...",
        "fill": "...",
        "size": "..."
      }
    }]
  }
}
```

Return ONLY valid JSON, no explanations.

A.4 An Example of the End-to-End Process

An end-to-end example of the Analytic Agent’s overall process (also see Figure 3)

For the query “Show weekly average handle time for the Seattle support team over the last quarter”:

(1) *The Orchestration Agent* detects a metric request with a temporal modifier.

(2) *The Target Selection Agent* maps “Seattle support team” to a unique `agent_group`.

(3) *The Database Querying Agent* chooses the `metrics/handle_time` endpoint, retrieves field definitions, computes the last-quarter window in the session timezone, validates the payload, executes with retries if needed, and normalizes results by ISO week.

(4) *The Visualization Agent* emits a line chart spec with week on the *x*-axis and average handle time on the *y*-axis, including a policy-compliant legend and a note about the partial current week if applicable.

A.3 Query Versatility and Examples

To demonstrate the versatility of the Analytic Agent, Table 3 illustrates its ability to parse diverse natural language queries. These examples cover a range of metrics (e.g., counts, rates), temporal granularities (e.g., relative dates, quarters), and entity-specific filters, mapping them to the appropriate internal reasoning components.

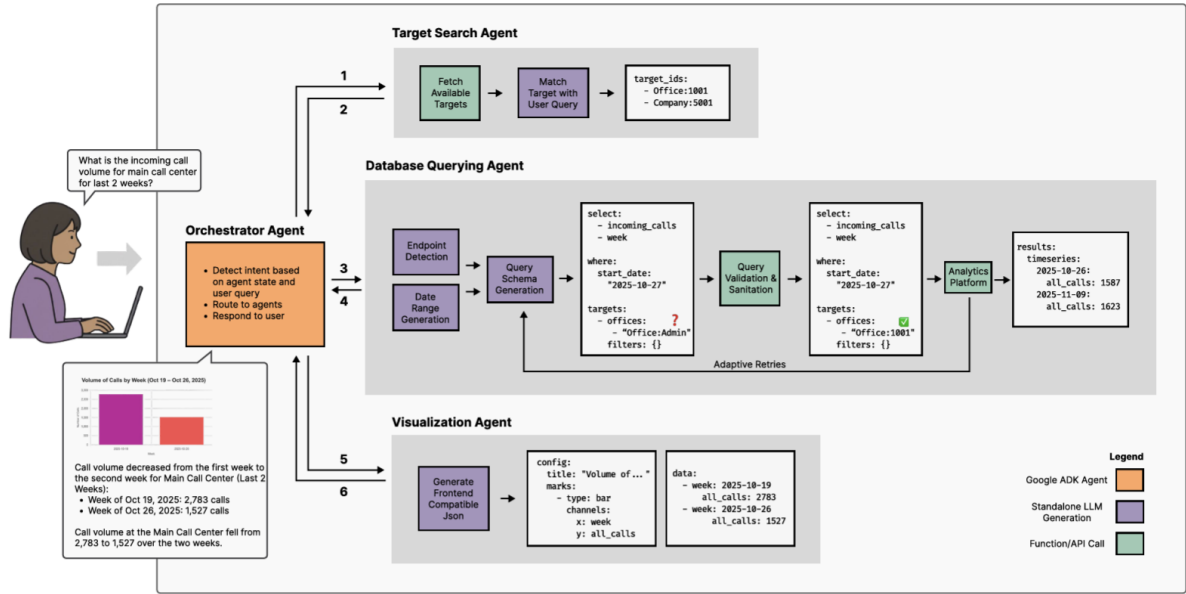


Figure 3: Demonstrating the end-to-end workflow diagram of the Analytic Agent system for an example query.

Natural Language Query	Key Metric(s) / Intent	Key Dimensions / Filters
Show me the average AI talk time percent for meetings at the Primary Office in Q1 2025 , broken down month by month.	avg:percent_ai_talk_time	targets: [Primary Office], group_by: [month], date_range: [Q1 2025]
What was the deflection rate for the main Support call center last month ?	deflection_rate	targets: [Support], date_range: [last month]
What was the hourly count of all voicemails for the Customer Care team yesterday ?	all_voicemails	targets: [Customer Care], group_by: [hour], date_range: [yesterday]
Show me the total texts received for the Services department this month . Break it down by group and by week.	inbound_texts	targets: [Services], group_by: [group, week], date_range: [this month]
Show me the average resolution time for digital sessions last week , broken down by channel.	average_resolution_time	group_by: [channel], date_range: [last week]
Show me the total number of internal meetings at the Primary Office in May 2025.	internal_meetings	targets: [Primary Office], date_range: [May 2025]
Show me the CSAT score for calls at the Primary Office over the last 4 weeks , broken down week by week.	average_csat_score	targets: [Primary Office], group_by: [week], date_range: [last 4 weeks]
How many surveys were completed each day for the main Support call center this week ?	survey_count	targets: [Support], group_by: [day], date_range: [this week]
How many handled calls resulted in a 'Sale Closed' disposition this month ?	handled_calls	dispositions: ['Sale Closed'], date_range: [this month]

Table 3: Example queries demonstrating the agent's parsing and reasoning versatility.